Smart Contract Security Audit (Round 2)

Scope: the on-chain pERC20 reference implementation under `contracts/`, re-audited on the **current source** after (a) the round-1 remediation of L-01…L-05 / I-01…I-04, and (b) a **major new change since round-1: the EIP-1167 clone factory + initializable `PERC20` / `OrchardVerifier`**.
Question answered: can the pERC20 privacy token be **maliciously forged, created, minted, transferred, or burned** by an attacker — and did the clone migration open any new hole?

---

1. Executive Summary

Round-1 confirmed that an **unprivileged attacker** has no path to forge, counterfeit, double-spend, or illegally mint/transfer/burn pERC20, and remediated all Low/Informational findings (L-01…L-05, I-01…I-04). This second round (a) **regression-verifies** those fixes are present and correct on the current source, and (b) audits the **new attack surface introduced since round-1** — the migration of `PERC20Factory` to **EIP-1167 minimal-proxy clones**, which required making `PERC20` / `OrchardVerifier` **initializable** (constructor → `initialize`, `issuer` `immutable` → storage, a one-shot `_initialized` guard, and the per-bundle `maxActions` cap moved out of an inline initializer).

Headline result. Still **no vulnerability that lets an unprivileged attacker forge, counterfeit, double-spend, or illegally mint / transfer / burn.** The round-1 layered defense is intact and the clone migration is implemented safely: the shared implementation is **locked** (initialized in the factory constructor, so its `_initialized` guard rejects any hijack), every clone is **initialized atomically inside `createPerc20`** (no front-running window), the one-shot guard blocks re-initialization, and the storage layout was preserved (`_allRootsEver` stays at slot 140, on which the no-mock E2E harness depends). All round-1 fixes (L-01…L-05, I-01…I-04) are confirmed in place. `forge test`: **76/76 pass** (up from 70; +6 clone-specific tests).

The one real bug the migration introduced — caught and fixed. Clones do **not** run the implementation's constructor, so any state variable set by a Solidity **inline initializer** silently stays zero in a clone. `OrchardVerifier.maxActions` was `= 10` inline; left unfixed, **every cloned pool would have `maxActions = 0` and revert *all* bundles** (`bundle exceeds maxActions`) — a total functional brick of every factory-deployed asset (a liveness/availability failure, not a theft vector). This was caught by the new clone tests and fixed by moving the assignment into `_initOrchardVerifier` (**L-06**, Resolved). An exhaustive sweep confirms `maxActions` was the only such hazard.

Where the real risk still is: privilege, not the protocol. Unchanged from round-1: each asset's `admin` can hot-swap the Groth16 verifier with no timelock/immutability (**H-01**, Open) and holds concentrated, un-timelocked powers (**M-01**, Open). The clone migration does **not** change this — each clone has independent storage, so `admin` / `groth16Verifier` are per-asset exactly as before; a per-asset admin-key compromise remains catastrophic for that asset only.

Findings summary

ID	Title	Severity	Status
H-01	Admin can hot-swap the Groth16 verifier with no timelock / immutability → counterfeit & drain	**High**	Open (unchanged; now per-clone)

ID	Title	Severity	Status
M-01	Concentrated, un-timelocked admin powers (verifier, frozen root, maxActions)	**Medium**	Open (unchanged)
L-06	Clone migration: `maxActions` inline initializer not run for EIP-1167 clones → every cloned pool bricks (reverts all bundles)	**Low** (availability)	**Resolved** (new)
I-05	Factory's locked implementation emits `Perc20Created` → discoverable-but-inert "phantom" pool	Informational	**Resolved** (documented) (new)
I-06	`issuer` changed `immutable` → write-once storage (clone requirement); compile-time immutability guarantee lost	Informational	**Open / accepted** (new)
L-01	Public EC points (`cv_net`, `rk`) consumed without on-chain on-curve/subgroup checks	Low	**Resolved** (regression-verified)
L-02	Schnorr malleability: `s` not range-checked	Low	**Resolved** (regression-verified)
L-03	`maxActions` upper bound too high (50)	Low	**Resolved** (regression-verified, ≤ 16)
L-04	Frozen-root update invalidates in-flight proofs (liveness)	Low	**Resolved** (regression-verified, opt-in grace)
L-05	Nullifier set written before the binding-signature check	Low	**Resolved** (regression-verified)
I-01	Note ciphertext not proven in-circuit → recipient non-receipt risk	Informational	**Resolved** (documented)
I-02	NUMS generators assumed prime-order without recorded check	Informational	**Resolved** (verified + test)
I-03	Token metadata not authenticated at `create`	Informational	**Resolved** (documented)
I-04	`pointNeg` non-canonical for `x = 0`	Informational	**Resolved**

2. Audited Files

File	LoC	Role	Δ since round-1
`contracts/orchardverifier/OrchardVerifier.sol`	396	Proof verification + note state machine (tree, nullifiers, roots, sig checks)	**initializable** (`_initOrchardVerifier` + `_initialized` guard; `maxActions` moved to init)
`contracts/ptoken/PERC20.sol`	174	Metadata, `totalSupply`, `mint` / `burn` / `transfer`, frozen-root setter	**initializable** (`initialize` added; `issuer` immutable → storage)

File	LoC	Role	Δ since round-1
`contracts/ptoken/PERC20Factory.sol`	92	EIP-1167 **clone** deployer (`createPerc20` / `createPerc20Deterministic`)	**rewritten** (clones, not `new PERC20`)
`contracts/proxy/Clones.sol`	63	EIP-1167 minimal-proxy library (`clone` / `cloneDeterministic` / `predict`)	**new**
`contracts/orchardverifier/ActionGroth16Verifier.sol`	52	Decode proof, recompute `pub_hash`, delegate to pairing verifier	—
`contracts/crypto/hash/ActionPubHash.sol`	56	On-chain `pub_hash` sponge (must match circuit)	—
`contracts/crypto/signature/BindingSignature.sol`	158	Value-conservation Schnorr	— (L-01/L-02 fixes present)
`contracts/crypto/signature/SpendAuthSignature.sol`	104	Per-action spend-authorization Schnorr	— (L-01/L-02 fixes present)
`contracts/crypto/merkle/IncrementalMerkleTree.sol`	115	Depth-32 Poseidon frontier tree	—
`contracts/crypto/curve/BabyJubJub.sol`	243	Twisted-Edwards EC ops + NUMS generators	— (I-02/I-04 fixes present)
`contracts/orchardverifier/Groth16ProofCodec.sol`	24	`abi.(en/de)code` of `(pA,pB,pC)`	—
`contracts/interfaces/{IEndpointCore,IPERC20,IActionGroth16Verifier}.sol`	—	Structs, events, layout	—

Relied upon, not re-derived: `Groth16PairingVerifier.sol` (snarkjs VK) and `PoseidonT3.sol` (generated permutation), exercised by the no-mock e2e suite.

---

3. Severity Classification

- **Critical** — directly leads to loss of funds / unauthorized supply, exploitable by any user.
- **High** — loss of funds or protocol integrity under a realistic (possibly privileged) actor.
- **Medium** — conditional or partial impact; defense-in-depth gaps exploitable when combined.
- **Low** — limited impact, hard-to-trigger, or non-financial (incl. availability/liveness).
- **Informational** — documentation / hygiene / latent assumptions, no direct impact.

---

4. System Overview & the (new) Clone-Deployment Attack Surface

The execution model is unchanged from round-1 (§4 there): a `PrivacyCall { bytes actions, uint256[3] bindingSig }` funnels through `mint`/`burn`/`transfer` into the internal `_executeBundle`, which — **before any state write** — range-checks the 8 `pubFields`, pins them to externally-meaningful state (`frozenRoot`, `cmx`, `anchor∈_allRootsEver`, `nfOld`), verifies the Groth16 proof (`pub_hash` recomputed via `ActionPubHash`, byte-identical to the circuit), the spend-auth Schnorr, and the binding Schnorr; only then consumes nullifiers (L-05) and inserts commitments. That analysis carries over verbatim and is regression-verified in §5.

New surface — how an asset is deployed (clones):

```
PERC20Factory(verifier)                       // constructor:
  └─ implementation = new PERC20("pERC20-implementation","pIMPL",0,address(this),verifier)
                                               //   ↳ runs the constructor →
_initOrchardVerifier
                                               //     sets _initialized=true ⇒
implementation LOCKED
createPerc20(name,symbol,decimals)             // per asset:
  ├─ pool = Clones.clone(implementation)       //   45-byte EIP-1167 proxy (CREATE /
CREATE2)
  └─ PERC20(pool).initialize(name,symbol,dec,  //   atomic, same tx ⇒ no front-run window
            msg.sender, verifier)              //   issuer = caller; emits Perc20Created
```

Attack questions specific to this surface, and why each fails:

- **Hijack a freshly-created clone by front-running its `initialize`.** Impossible: `createPerc20` clones *and* initializes in **one transaction**; there is no inter-tx window. A clone created out-of-band by an attacker is *their own* contract (they pay for it and set themselves issuer) — it cannot touch anyone else's asset.
- **Re-initialize a live clone to seize `admin`/`issuer`/verifier.** Blocked by the one-shot `_initialized` guard in `_initOrchardVerifier` (`AlreadyInitialized`); verified by `test_clone_initialize_is_one_shot`.
- **Initialize / hijack the shared implementation to compromise all clones.** Two independent reasons it fails: (1) the implementation is **already initialized** (its constructor ran), so `initialize` reverts on it (`test_factory_implementation_is_locked`); (2) even if it weren't, EIP-1167 clones have **independent storage** — they do not delegate-read the implementation's storage — so the implementation's state cannot affect any clone (`test_clones_have_independent_state`).
- **Brick all clones via the missing constructor.** This was the **one real defect** (L-06) — `maxActions` would default to 0 in clones — found and fixed before it could ship.
- **Forge `create` / spoof identity.** Unchanged (I-03): identity is the contract address; binding sighashes embed `address(this)`. A clone's `Perc20Created` is emitted in the clone's own delegatecall context, so `address(this)` is the clone address — indexers see the correct address.

---

5. Findings

[H-01] Admin can hot-swap the Groth16 verifier — High — Open (unchanged; now per-clone)

Unchanged from round-1. `OrchardVerifier.setGroth16Verifier(address)` is `onlyAdmin`, immediate, no timelock/immutability. A malicious/compromised admin can install a verifier that accepts any proof and counterfeit/drain that asset. **Clone-migration note:** each clone has its own `groth16Verifier` storage slot and its own `admin`, so this is strictly **per-asset** — compromising one asset's admin does not affect sibling clones from the same factory. The factory's own `groth16Verifier` is `immutable` and is only *read* at clone-init time; the factory cannot rotate a deployed clone's verifier. **Recommendation (unchanged):** make the verifier immutable or timelocked, and place `admin` behind a multisig/timelock.

[M-01] Concentrated, un-timelocked admin powers — Medium — Open (unchanged)

Unchanged from round-1. A single `admin` per asset controls `setGroth16Verifier` (H-01), `setFrozenRoot`, and `setMaxActions`; `issuer` separately holds unlimited `mint`. The clone migration does not alter this (per-clone storage). **Recommendation:** multisig/timelock for `admin`; separate `admin` from `issuer`; per-power roles.

[L-06] Clone migration: inline state-var initializer (`maxActions`) not executed for EIP-1167 clones — Low (availability) — Resolved (new)

Description. EIP-1167 clones delegate-call a shared implementation and **never run its constructor**; Solidity *inline* state-variable initializers (`uint256 public maxActions = 10;`) are compiled into the constructor, so a clone gets the type-zero default instead. `OrchardVerifier._verifyBundle` requires `n <= maxActions`; with `maxActions = 0`, **every** bundle reverts `"bundle exceeds maxActions"`.

Impact. Availability, not security: **every factory-deployed (cloned) pool would be completely non-functional** — no mint, transfer, or burn could ever execute (all revert). Standalone `new PERC20(...)` was unaffected (it runs the constructor). No funds at risk; no forgery/theft. (Caught by the new clone E2E test `test_E2E_factory_clone_creates_pool_and_mints`, which performs a real-proof mint against a clone and would otherwise have reverted.)

Resolution (fixed). Removed the inline initializer (`uint256 public maxActions;`) and set `maxActions = 10` inside `_initOrchardVerifier` (`OrchardVerifier.sol:136`), so both deploy paths (standalone constructor and clone `initialize`) set it. An **exhaustive sweep** of all state-var declarations in `OrchardVerifier` + `PERC20` confirms `maxActions` was the *only* inline-initialized state variable; every other field (`frozenRootGracePeriod`, `_frozenRoot`, `_prevFrozenRoot`, `pendingAdmin`, `_rootsPushed`, `_totalSupply`, ...) correctly relies on the zero default, which is the intended initial value for a fresh pool.

Recommendation. Add a lint/CI guard (or a code comment convention) that no `OrchardVerifier`/`PERC20` state variable may use an inline initializer — all initial state must flow through `_initOrchardVerifier`/`initialize`, since clones bypass the constructor.

Status: Resolved.

[I-05] Factory's locked implementation is a discoverable-but-inert "phantom" pool — Informational — Resolved (documented) (new)

Description. The factory deploys its implementation via `new PERC20("pERC20-implementation","pIMPL",0,address(this),verifier)` in its constructor, which runs `_initPerc20Metadata` and therefore **emits** **`Perc20Created(implementation, factory, "pERC20-implementation", "pIMPL", 0)`**. An indexer that auto-discovers assets by scanning `Perc20Created` chain-wide (the project's indexer does exactly this) will index the implementation as an asset.

Impact. Informational. The implementation is **inert**: `mint` is `onlyIssuer` = the factory, which never calls it; its note tree is empty; and although `transfer`/`burn` are permissionless, there are no notes to spend, so at most a griefer could insert zero-value dummy-spend notes into a throwaway contract holding no value. It is not a counterfeit/theft vector. The concern is purely that wallets/indexers may surface a bogus "pERC20-implementation" asset.

Resolution (documented). `PERC20Factory` exposes `implementation()` (public immutable) and its NatSpec instructs indexers to **filter `factory.implementation()`**. Recommendation for completeness: have the indexer skip any `Perc20Created` whose pool equals a known factory `implementation()`, and/or skip `issuer == <factory address>`.

Status: Resolved (documented). *(Optional hardening, not required: give the implementation a constructor mode that locks initializers without emitting `Perc20Created` — at the cost of splitting the standalone constructor.)*

[I-06] `issuer` downgraded from `immutable` to write-once storage — Informational — Open / accepted (new)

Description. Round-1's Positive Observations listed "**immutable `issuer`**". The clone migration required converting `issuer` to a storage variable (clones cannot use `immutable`, which is baked into the implementation's bytecode and shared by all clones). It is now set once in `_initPerc20Metadata` (guarded by `_initialized`) with **no setter**, so it is *functionally* write-once.

Impact. Informational. There is no path to mutate `issuer` after initialization today. The only loss is the **compile-time** guarantee: `immutable` made "issuer can never change" un-bypassable even by a future code change, whereas a storage field relies on the ongoing absence of a setter. (`admin`, by contrast, is *intentionally* mutable via the two-step transfer; `issuer` is not.)

Recommendation. Keep `issuer` setter-free; add a comment marking it write-once-after-init, and a test asserting no function writes `issuer` outside initialization (mirrors the L-06 lint suggestion).

Status: Open / accepted (inherent to the clone design).

[L-01]…[L-05], [I-01]…[I-04] — regression-verified Resolved

Re-read on the current source; all round-1 fixes are present and correct:

- **L-01** — `BindingSignature.verify` `isOnCurve`-checks every `cv_net` (`BindingSignature.sol:127,130`); `SpendAuthSignature.verify` `isOnCurve`-checks `rk` (`SpendAuthSignature.sol:84`) — before any EC arithmetic. ✓
- **L-02** — both libraries reject `sigS >= SUBGROUP_ORDER` (`BindingSignature.sol:106`, `SpendAuthSignature.sol:81`). ✓
- **L-03** — `setMaxActions` enforces `1 ≤ limit ≤ 16` (`OrchardVerifier.sol:194`). ✓
- **L-04** — opt-in grace window: `_setFrozenRoot` records `_prevFrozenRoot` + `_frozenRootUpdatedAt`; `_verifyAction` accepts the previous root within `frozenRootGracePeriod` (default 0 = strict) (`OrchardVerifier.sol:314-323`). ✓ *(For clones, `frozenRootGracePeriod`/`_prevFrozenRoot` correctly default to 0 — strict mode — which is the safe default; see L-06 sweep.)*
- **L-05** — `_verifyBundle` is `view` and writes nothing; `_executeBundle` verifies the binding signature, **then** consumes nullifiers (`OrchardVerifier.sol:261-282`). The "no state mutation before value conservation" invariant holds literally (`test_failed_binding_leaves_state_clean`). ✓
- **I-01** — note-receipt trust boundary documented in `IEndpointCore.NoteAdded`. ✓
- **I-02** — generators recorded prime-order in `BabyJubJub.sol:44-47` and asserted on-curve/non-identity by `test/Generators.t.sol`. ✓
- **I-03** — metadata-not-authenticated documented in `PERC20Factory.createPerc20` NatSpec (identity =

address + issuer). ✓
- **I-04** — `pointNeg(0,y)` returns `0` (canonical), not `Fr` (`BabyJubJub.sol:84`). ✓

---

6. Positive Observations (clone migration)

Beyond round-1's positives (no public `bundle()`, canonical-field guard, tight field binding, value/authority signatures, replay resistance, anchor safety, supply guards, matched-set `ActionPubHash`), this round confirms the **clone migration is implemented safely**:

- **Implementation is locked** — deployed and fully initialized in the factory constructor, so `_initialized = true` makes `initialize` revert on it (no uninitialized-implementation hijack; verified `test_factory_implementation_is_locked`).
- **Atomic clone+initialize** — `createPerc20` performs `Clones.clone` then `initialize` in one transaction; no front-running window between proxy creation and initialization.
- **One-shot init guard** — `_initialized` blocks re-initialization (`test_clone_initialize_is_one_shot`).
- **Independent per-clone storage** — each asset has its own tree, nullifier set, supply, metadata, `admin`, `groth16Verifier`, and `cmxFrozenRoot` (`test_clones_have_independent_state`); only the immutable verifier *reference value* is shared at init time.
- **Storage layout preserved** — `_initialized` appended last in `OrchardVerifier`, so `_allRootsEver` remains at slot 140 (the no-mock E2E harness injects anchors against fixed slots); 20/20 real-proof E2E cases pass against clones and standalone deployments alike.
- **Deterministic addresses are caller-bound** — `createPerc20Deterministic` salts with `keccak256(msg.sender, salt)`, so distinct issuers cannot collide on or front-run each other's predicted addresses; `predictPerc20Address` matches (`test_factory_deterministic_address_matches_prediction`).
- **Factory has no governance surface** — `groth16Verifier` and `implementation` are `immutable`; the factory holds no admin role and no upgradability.

---

7. Conclusion

For an **unprivileged attacker**, this round confirms **no path to forge, counterfeit, double-spend, or illegally mint / transfer / burn** pERC20. All round-1 remediations (L-01…L-05, I-01…I-04) are present and correct, and the **EIP-1167 clone migration — the main change since round-1 — is implemented safely**: locked implementation, atomic and one-shot initialization, independent per-clone storage, and a preserved storage layout. The migration's one real defect (**L-06**: a missing constructor would have left `maxActions = 0` and bricked every cloned pool — an availability, not a security, failure) was caught by the new clone tests and fixed by moving initialization into `_initOrchardVerifier`; an exhaustive sweep confirms no other inline-initializer hazard. The two new Informational items (**I-05** phantom implementation pool; **I-06** `issuer` immutable→storage) are inherent to the clone design and documented/accepted.

The material risk remains **centralization**, unchanged from round-1: per-asset `admin` can hot-swap the Groth16 verifier with no timelock (**H-01**) and holds concentrated powers (**M-01**); a per-asset admin-key compromise can counterfeit and drain that asset.

Priority recommendations before mainnet (unchanged from round-1, still Open):

1. Make the Groth16 verifier **immutable or timelocked**, and place `admin` behind a **multisig/timelock** (H-01, M-01).
2. Add a **CI guard** that no `OrchardVerifier`/`PERC20` state variable uses an inline initializer (L-06 class), and a test asserting `issuer` has no post-init writer (I-06).
3. Have the **indexer filter `factory.implementation()`** (I-05).
4. Record the **prime-order generator** verification and a **matched-set CI** test (I-02; circuit Suggestions 4/9); commission a **dedicated review of the signature + Poseidon libraries** and a **formal Groth16 MPC ceremony** (circuit Suggestion 6).

---

8. Disclaimer

This review covers the Solidity contracts listed in §2 at their current state (post round-1 remediation + clone migration). `Groth16PairingVerifier` (snarkjs VK) and `PoseidonT3` (generated permutation) are trusted, not re-derived; the Baby JubJub EC and Schnorr libraries were read for correctness but not formally verified. Cross-layer soundness relies on the companion circuit audits (`docs/audit-circuits-action-{0,1,2-en,3-en,4-en}.md`); the EIP-1167 `Clones` library is a minimal in-repo port of the well-known OpenZeppelin implementation. The no-mock E2E suite (real Groth16 proofs for mint/transfer/burn verifying on-chain, against both clones and standalone deployments) provides strong evidence of correctness but is not a substitute for formal verification or a trusted-setup ceremony. An audit is not a guarantee of bug-freedom.